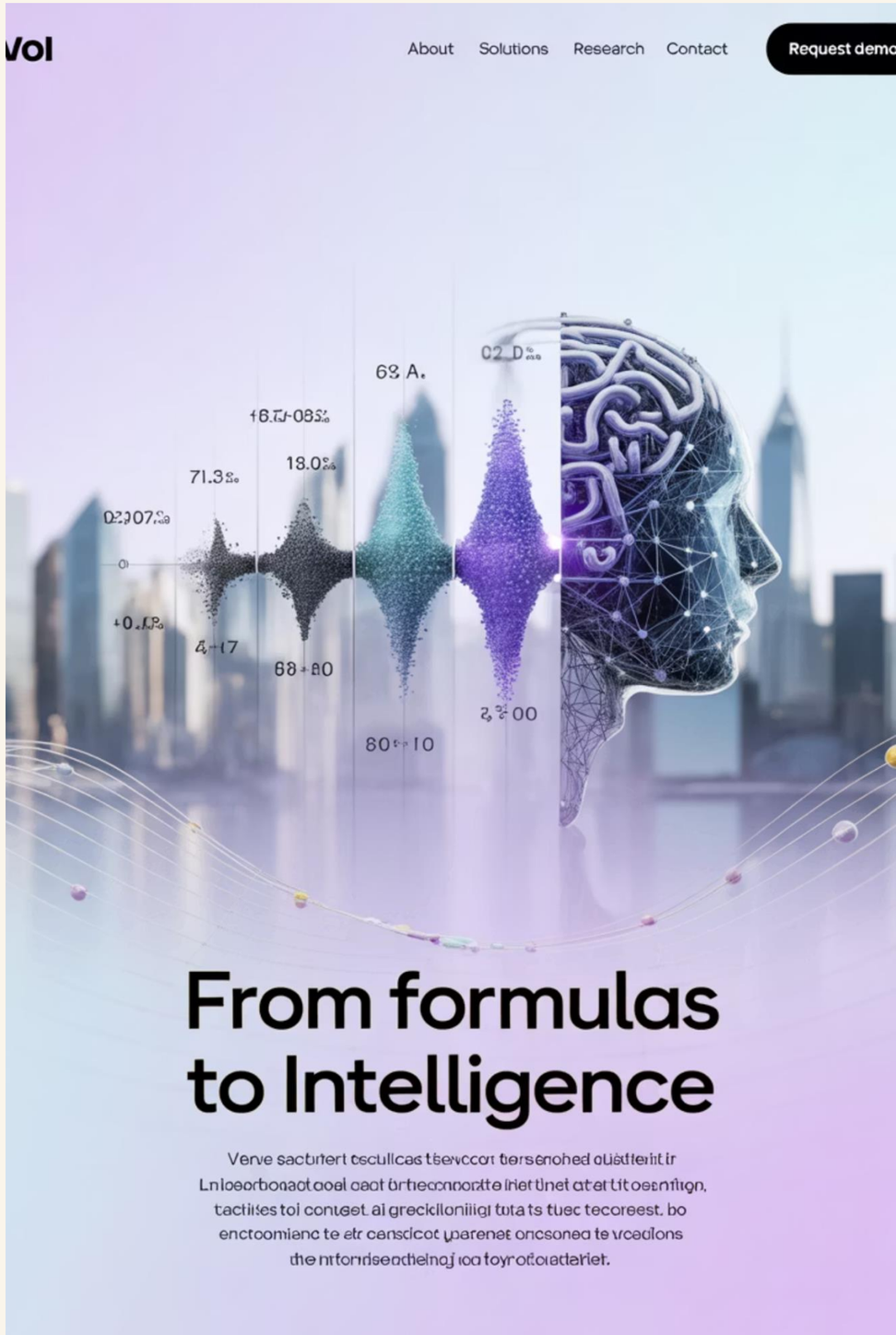
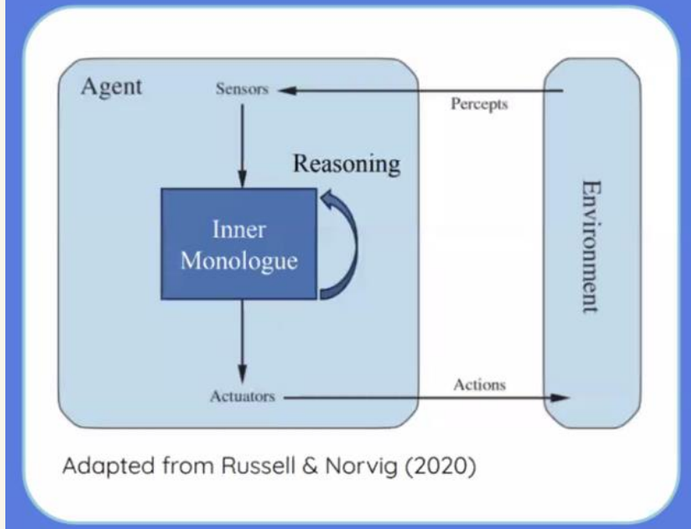
Agent in Finance

From mathematical models to generative AI

Andy Li, PhD, CFA, FRM



What Is agent?



Adapted from Russell & Norvig (2020)

Perception: Multimodal inputs including, text, image, audio, video, touch, etc.

Planning (Inner Monologue): Chain-of-Thought reasoning

Reflection: meta-reasoning in every stop

Actions: function/tool calling, embodied actions.



Bill Gates

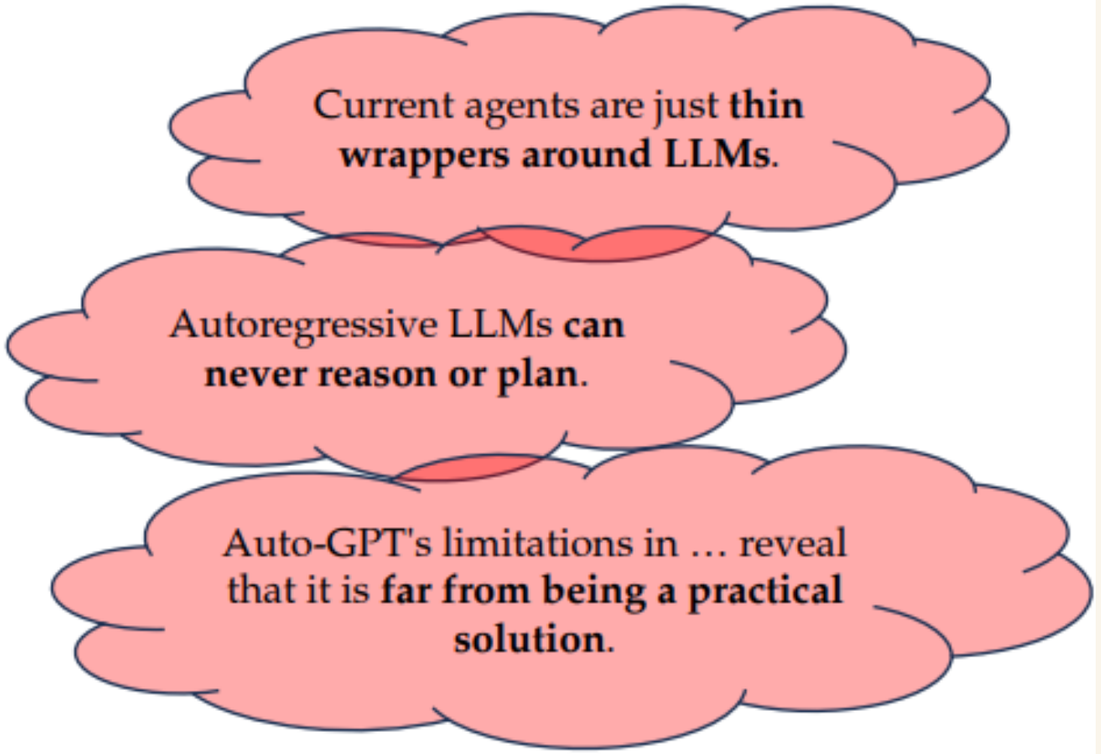
Agents are bringing about the **biggest revolution in computing** since we went from typing commands to tapping on icons.

Andrew Ng

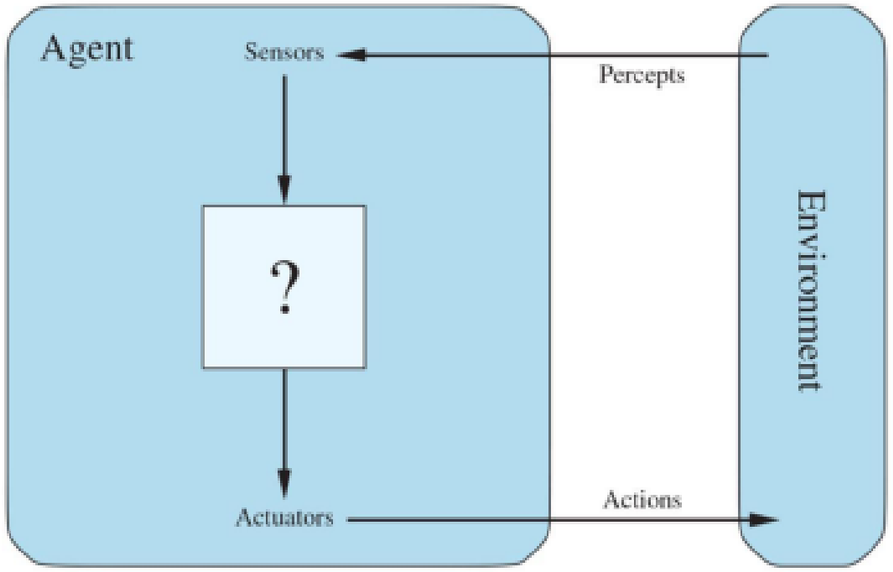
I think AI agentic workflows will drive **massive AI progress** this year.

Sam Altman

2025 is when **agents will work**.



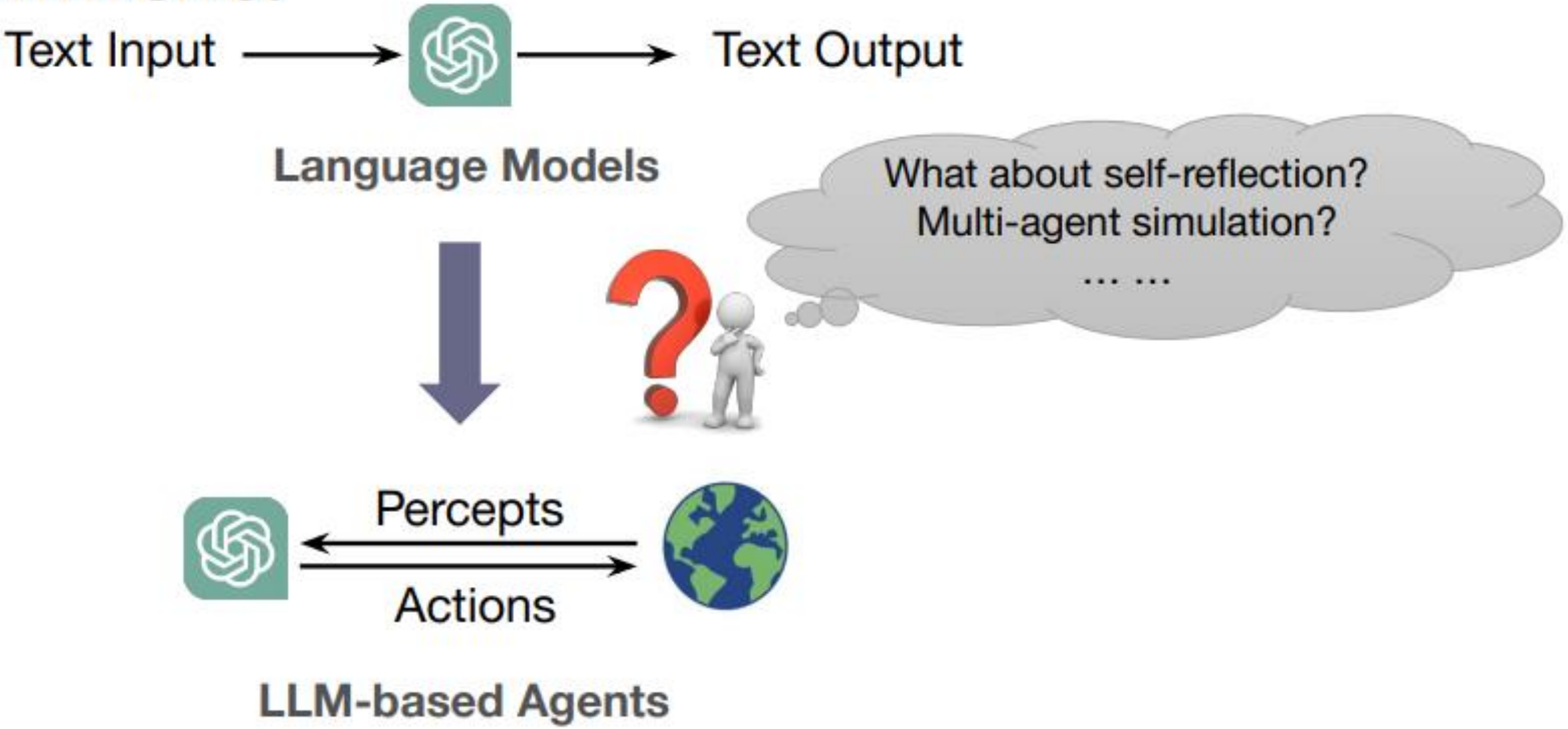
Why agents again?



"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."

-- Russell & Norvig, *AI: A Modern Approach* (2020)

‘Modern’ agent = LLM + external environment?



Two competing views

LLM-first view: We make an LLM into an agent!

- Implications: scaffold on top of LLMs, prompting-focused, heavy on engineering

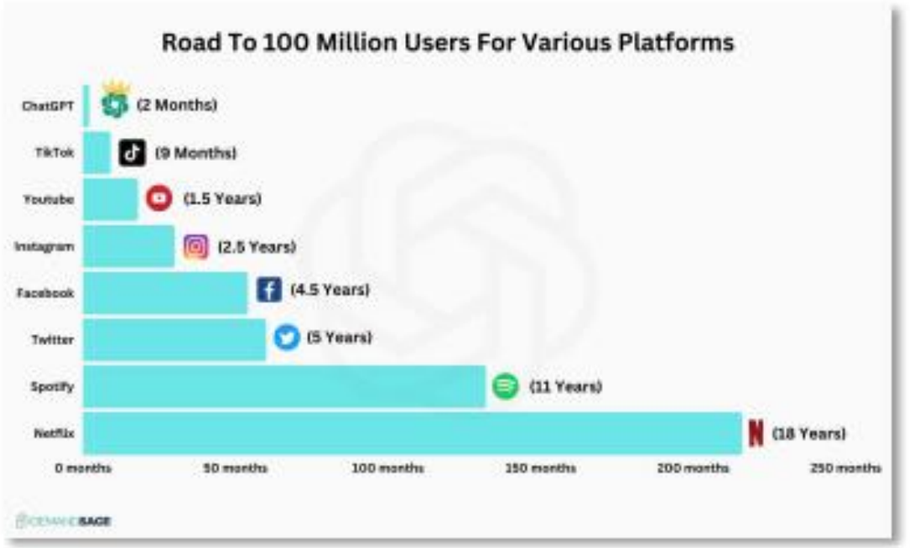
Agent-first view: We integrate LLMs into AI agents so they can use language for reasoning and communication!

- Implications: All the same challenges faced by previous AI agents (e.g., perception, reasoning, world models, planning) still remain, but we need to **re-examine them through the new lens of LLMs** and tackle new ones (e.g., synthetic data, self-reflection, internalized search)

What's fundamentally different now?

Contemporary AI agents, with integrated LLM(s), can *use language as a vehicle for reasoning and communication*

- Instruction following, in-context learning, output customization
- Reasoning (for better acting): state inferences, self-reflection, replanning, etc.



https://www.demandsage.com/chatgpt-statistics/



https://www.reddit.com/r/ChatGPT/comments/16jvl4x/wait_actually_yes/

A generalized notion of ‘reasoning’

Unlike humans, LLMs (mostly) only have one mechanism (token generation) for perception, intuitive inferences, and symbolic reasoning; everything is effortful and takes a forward pass



GPT-4o

In this image, we see a young boy sitting on the floor, looking down at a pile of broken dishes. He appears to be sad or remorseful. In front of him, a woman, likely his mother or caregiver, is kneeling and looking at him with a concerned, questioning expression. Her body language, with her hands open and her facial expression, suggests that she's asking him something or expressing surprise or frustration about the broken dishes.

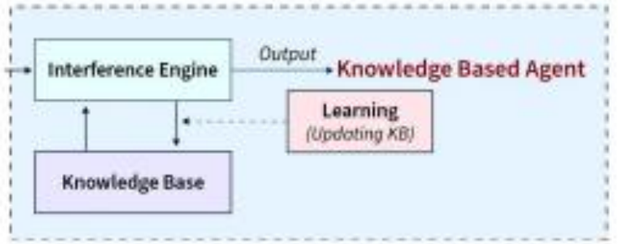
Perception

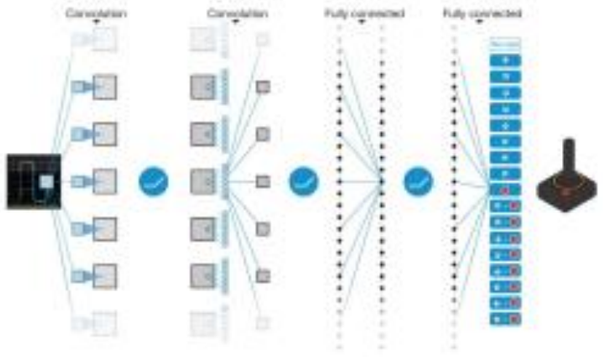
Intuitive Inference

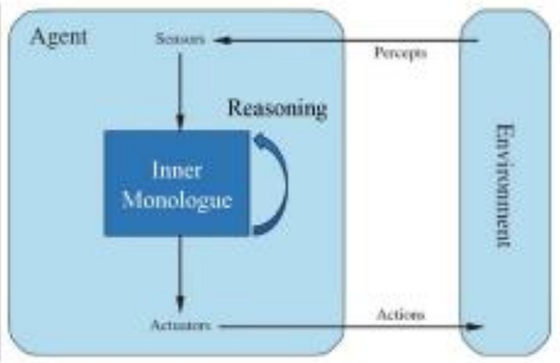
Reasoning

One may alternatively call this ‘thought’ to avoid the over-loaded term of ‘reasoning,’ at the risk of further anthropomorphizing machines

Evolution of AI agents





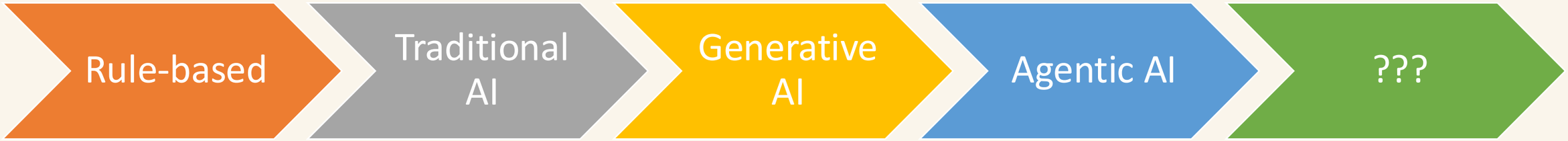


Logical Agent | **Neural Agent** | **Language Agent**

Expressiveness	Low bounded by the logical language	Medium anything a (small) NN can encode	High almost anything, esp. verbalizable parts of the world
Reasoning	Logical inferences sound, explicit, rigid	Parametric inferences stochastic, implicit, rigid	Language-based inferences fuzzy, semi-explicit, flexible
Adaptivity	Low bounded by knowledge curation	Medium data-driven but sample inefficient	High strong prior from LLMs + language use

Image sources: https://www.scaler.com/topics/artificial-intelligence-tutorial/knowledge-based-agent/,
Mnih et al., "Human-level control through deep reinforcement learning." Nature (2015)

Evolution of AI in Finance



Predefined strategies
Statistical Arbitrage

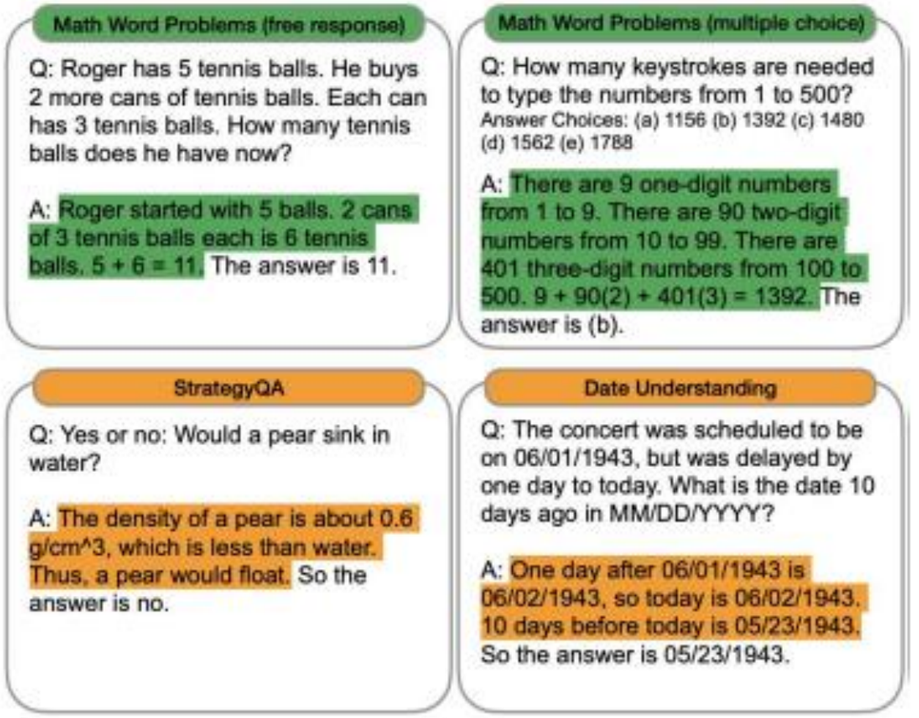
Machine Learning,
NLP, Sentiment Analysis

Text Summarization,
Video Generation
AI Coding

Use Tools, Agent-in-the Loop
Reasoning, Observation

Reasoning

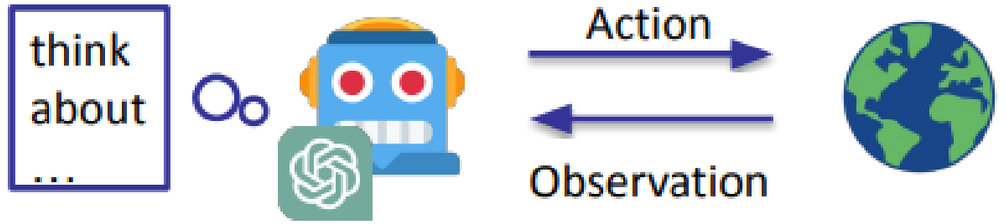
- For humans: various mental processes
- For LMs: intermediate generation
  - Imitate various (but not all) human mental processes



*(Wei et al., 2022), also see (Ling et al., 2017; Cobbe et al., 2021; Nye et al., 2021)

Reasoning

- For humans: various mental processes
- For LMs: intermediate generation
- For agents: internal actions
  - Why is it helpful?
  - Why is it special?
  - Why is it now?



Why is reasoning helpful for agents?

.....

Obs t

You are cooking a dish and seeing salt is out...

Reasoning

"The dish should be savory, and since salt is out, I should find the soy sauce instead. It is in the cabinet to my right..."

Act t

Turn right

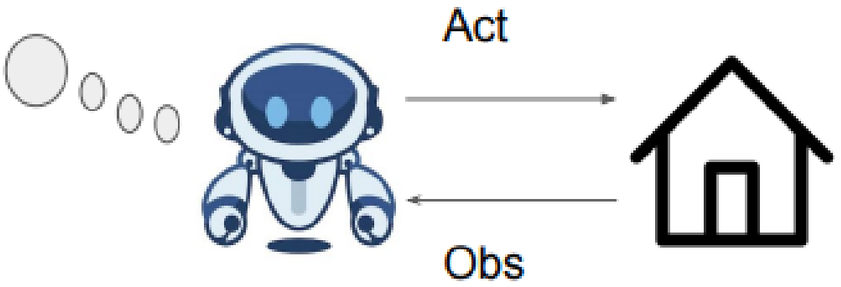
Obs t+1

You see a cabinet and a table...

Act t+1

Open cabinet

.....



Reason to act

1. Generalization
2. Alignment

Observation to Action mapping could be hard

Reasoning with acting

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program ……(long text omitted)

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports','Front Row (software)', …]

Thought 3: Front Row is not found. I need to searchFront Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software (long text omitted)

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keysSo the answer is keyboard function keys.
Act 4: Finish[keyboard function keys]



Why is reasoning just now for agents?

- Bigger action space -> More capacity, harder decision making
  - The space of reasoning/language is **infinite**
- LLMs learn reasoning priors by imitating various human reasoning traces

ReAct: action space $\hat{A} = A \cup \mathcal{L}$ augmented by reasoning



- $\hat{a}_t \in \mathcal{L}$ can be any language sequence
- Agent context $c_{t+1} = (c_t, \hat{a}_t, a_t, o_{t+1})$
- $\hat{a}_t \in \mathcal{L}$ only updates **internal context**

Why Is Multi-Agent Preferred

- Single agent is not strong enough!

- An easy-to-use prompting for effectiveness
- Test of time compute
  - Parallel & easy to scale
- Different agents represent different expertise
- Decentralized control & Privacy-persevering
  - Maintain data access control

Key Principles of Multi-Agent System

- ➢ Agent initialization
  - ➢ Profiling via persona or expertise
- ❑ Orchestration process
  - ❑ Coordinate different agents
- ❑ Agent team optimization
  - ❑ Optimize an existing agent team or agent selection

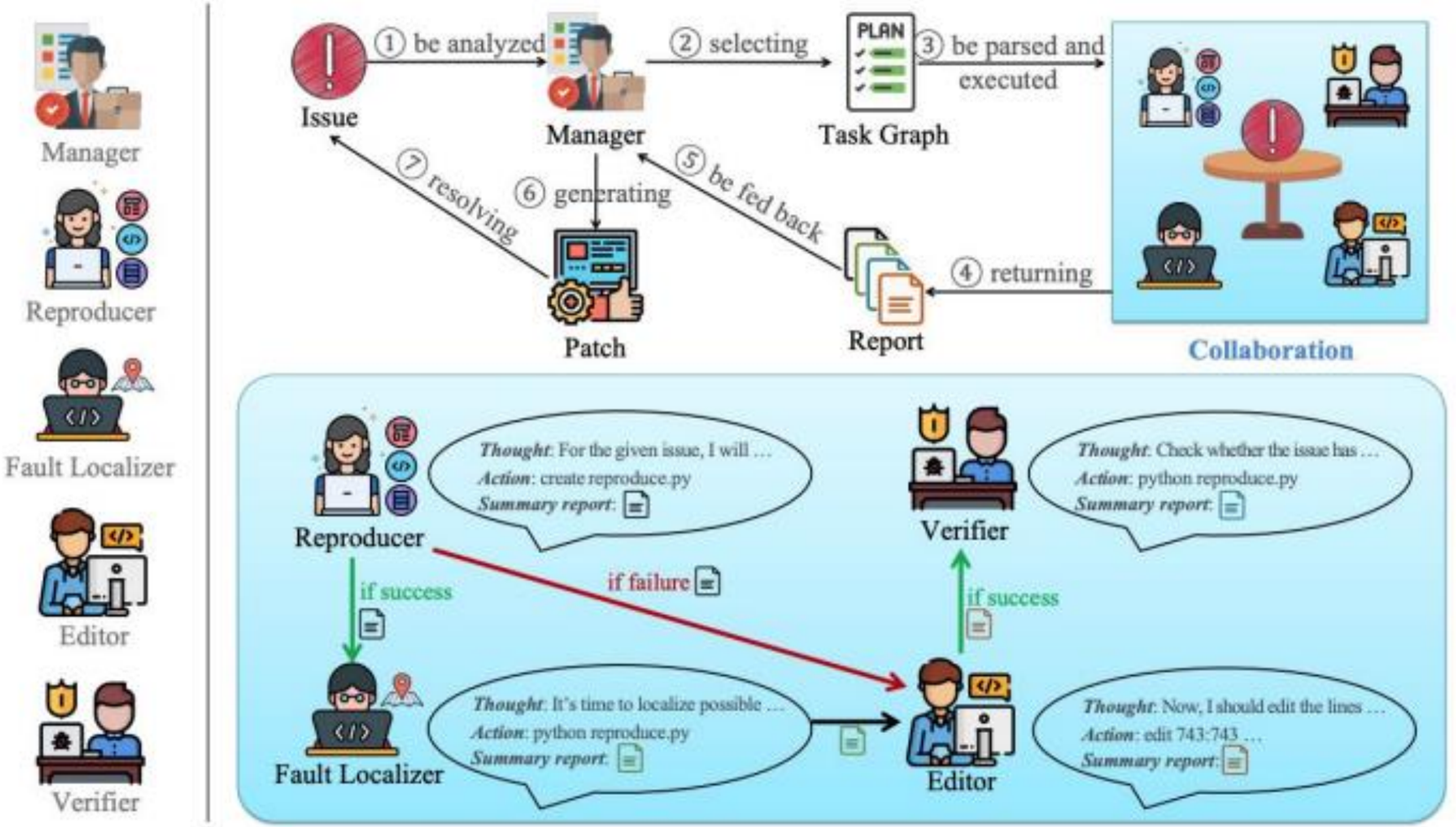
Agent Initialization via Persona Description



John Lin is a pharmacy shopkeeper at the Willow Market and Pharmacy who loves to help people. He is always looking for ways to make the process of getting medication easier for his customers; John Lin is living with his wife, Mei Lin, who is a college professor, and son, Eddy Lin, who is a student studying music theory; John Lin loves his family very much; John Lin has known the old couple next-door, Sam Moore and Jennifer Moore, for a few years; John Lin thinks Sam Moore is a kind and nice man; John Lin knows his neighbor, Yuriko Yamamoto, well; John Lin knows of his neighbors, Tamara Taylor and Carmen Ortiz, but has not met them before; John Lin and Tom Moreno are colleagues at The Willows Market and Pharmacy; John Lin and Tom Moreno are friends and like to discuss local politics together; John Lin knows the Moreno family somewhat well — the husband Tom Moreno and the wife Jane Moreno.

Park, Joon Sung, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. "Generative agents: Interactive simulacra of human behavior." In Proceedings of the 36th annual acm symposium on user interface software and technology, pp. 1-22. 2023.

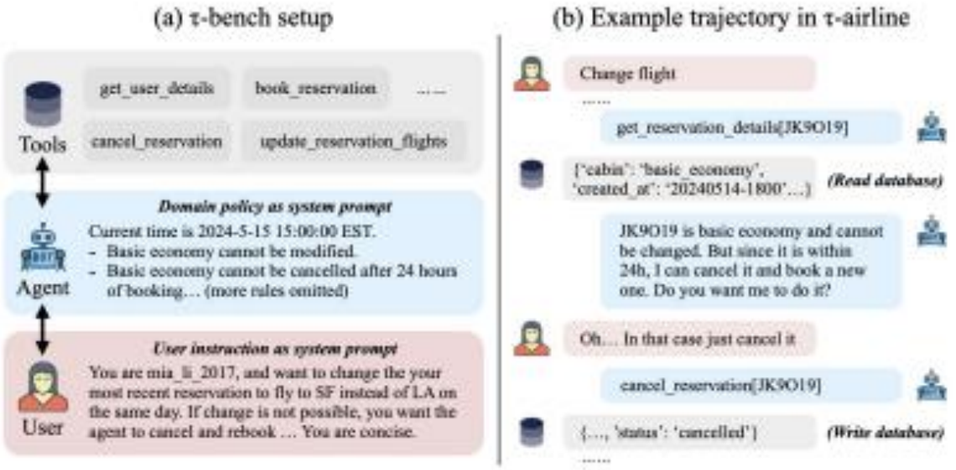
Agent Initialization via Roles and Actions



Coding agents

API/functional calling for tool use

- Environment: software systems such as databases, app/web services...
- Observation space: API docs, system info, error messages and logs...
- Action space: function calls, error handling routines...





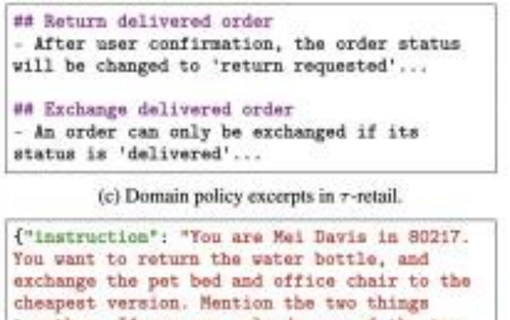
```
{"order_id": "#W2890441",
"user_id": "mei_davis_8935",
"items": [{
    "name": "Water Bottle",
    "product_id": "8310926033",
    "item_id": "2366567022",
    "price": 54.04,
    "options": {
        "capacity": "1000ml",
        "material": "stainless
        steel",
        "color": "blue"
    }}, ...], ...}
```

(a) An orders database entry in τ-retail.

```
def return_delivered_order_items(
    order_id: str,
    item_ids: List[str],
    payment_method_id: str,
) -> str: ...

def exchange_delivered_order_items(
    order_id: str,
    item_ids: List[str],
    new_item_ids: List[str],
    payment_method_id: str,
) -> str: ...
```

(b) An API tool in τ-retail.



```
## Return delivered order
- After user confirmation, the order status
will be changed to 'return requested'...

## Exchange delivered order
- An order can only be exchanged if its
status is 'delivered'...
```

(c) Domain policy excerpts in τ-retail.



```
{"instruction": "You are Mei Davis in 80217.
You want to return the water bottle, and
exchange the pet bed and office chair to the
cheapest version. Mention the two things
together. If you can only do one of the two
things, you prefer to do whatever saves you
most money, but you want to know the money
you can save in both ways. You are in debt
and sad today, but very brief.",
"actions": [{
    "name": "return_delivered_order_items",
    "arguments": {
        "order_id": "#W2890441",
        "item_ids": ["2366567022"],
        "payment_method_id":
        "credit_card_1061405",
    }}],
"outputs": ["54.04", "41.64"]}
```

(d) User instruction ensures only one possible outcome.

Gaming agents

Digital games

- Environment: game worlds/levels...
- Observation space: screenshots of game states, inventory, location...
- Action space: game controls (e.g., drop, move, attack, resource management...)





Robotic agents

Robotics for physical interaction

- Environment: physical world spaces
- Observation space: visual input, sensor readings, physical states, proprioception...
- Action space: motor controls (e.g., move, grasp, manipulate...)



More agent evaluation metrics

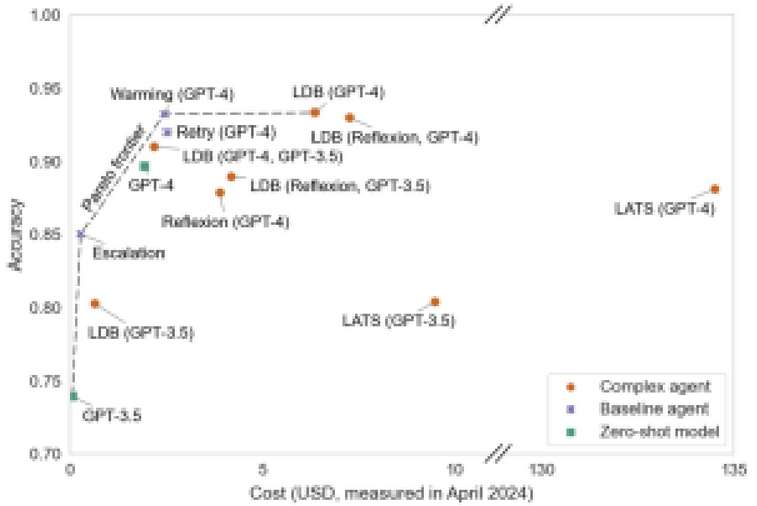
Other agent evaluation metrics

- Latency - efficiency
  - Compute aware success rate
  - Real time evaluation
- Robustness
  - Generalization to unseen domains, tasks, apps

AI Agents That Matter

Sayash Kapoor*, Benedikt Stroebl*, Zachary S. Siegel, Nitya Nadgir, Arvind Narayanan

Princeton University
July 2, 2024



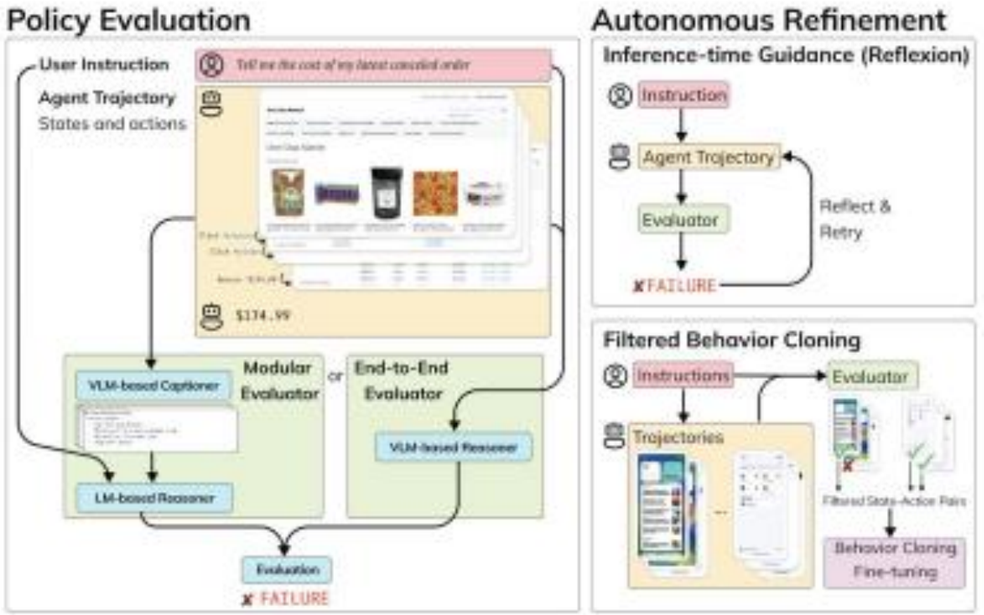
Limitations of agent benchmarks

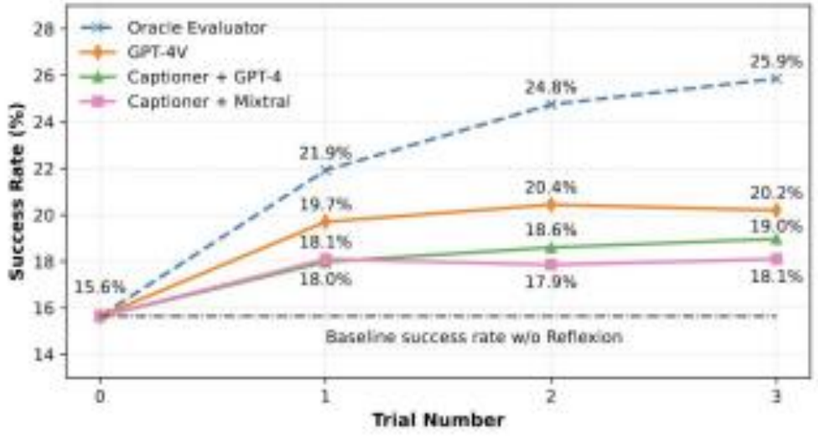
- Only can write evaluation scripts for very limited tasks, time-consuming
- Cannot script evaluation metrics for open-answer tasks

Possible solution: leveraging LLM/VLMs to automatically evaluate agent tasks?

Automatic agent evaluation

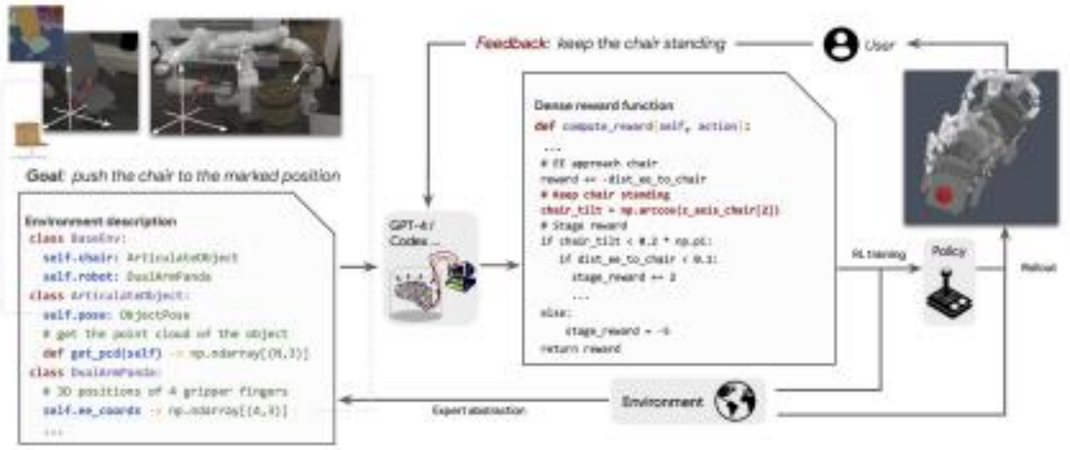
Automatically evaluate user instructions and arbitrary agent trajectories with LLM/VLMs

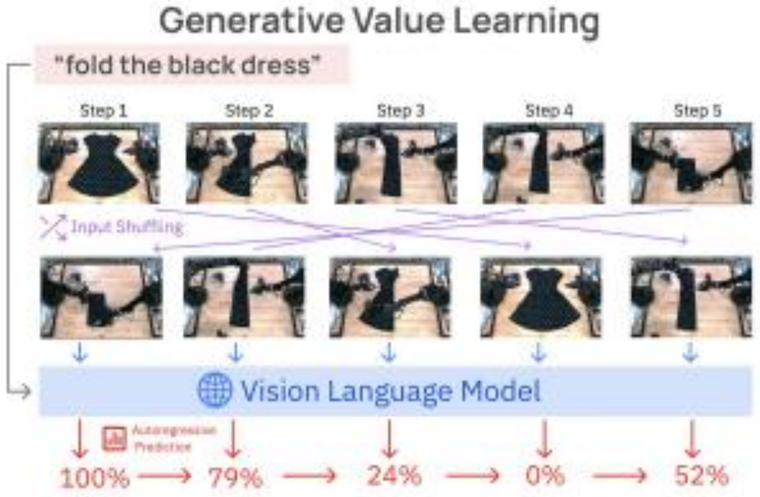




Automatic agent evaluation

- Leveraging coding ability of LLMs to automatically generate reward functions
- Leveraging the world knowledge embedded in VLMs to evaluate task progress





Real-World Applications

HR & Recruitment

- *Automated resume screening and candidate matching*
- *Skills gap analysis across applications*
- *Interview question generation based on resume content*

Education

- *Automated grading of essays and subjective questions*
- *Personalized feedback generation for students*
- *Course material adaptation based on performance*

Administrative Automation

- *Email classification, prioritization, and response drafting*
- *Document tagging and organization*
- *Meeting summarization and action item extraction*

Customer Support

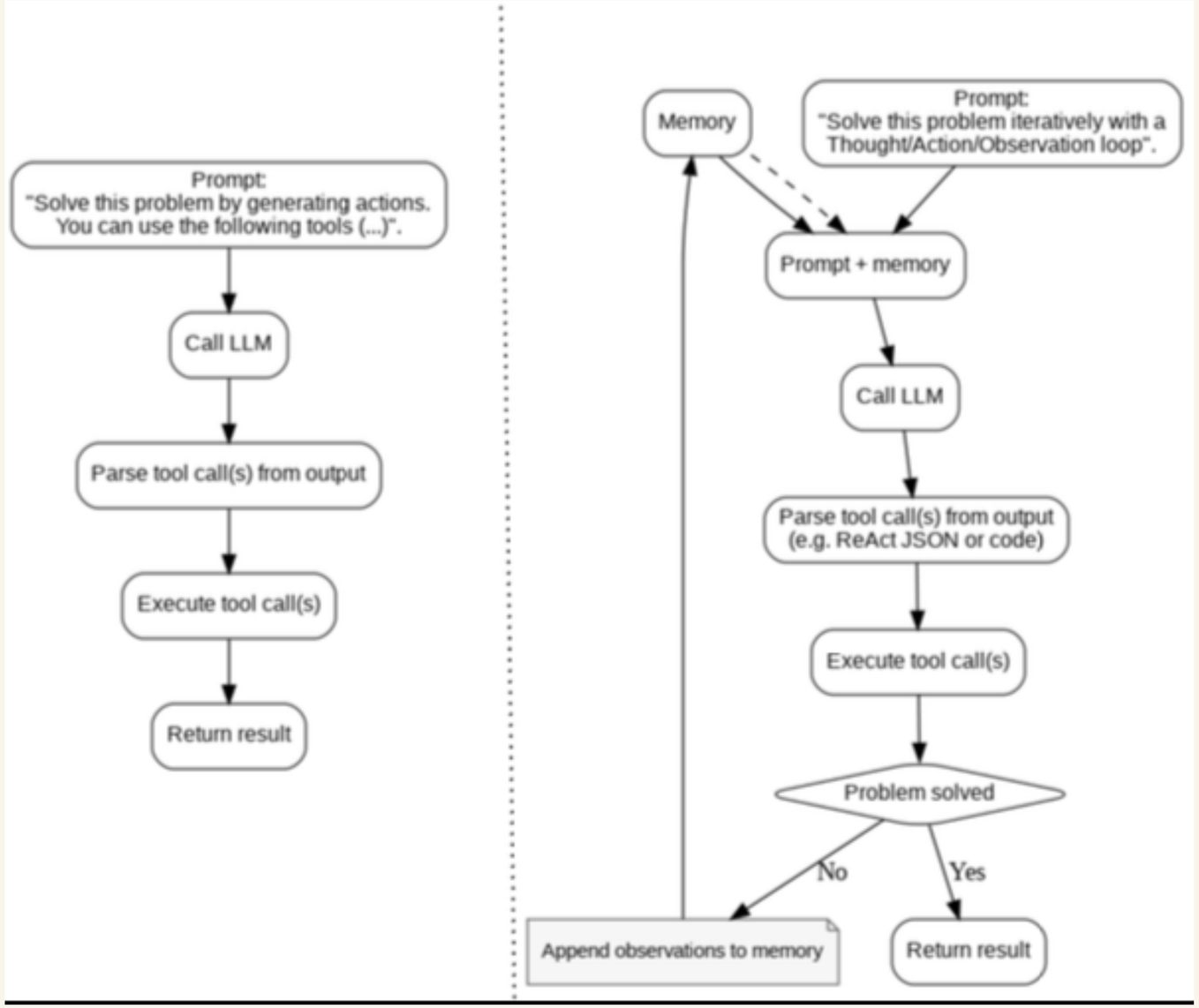
- *Ticket categorization and routing*
- *Response drafting with technical accuracy*
- *Knowledge base integration for context-aware answers*

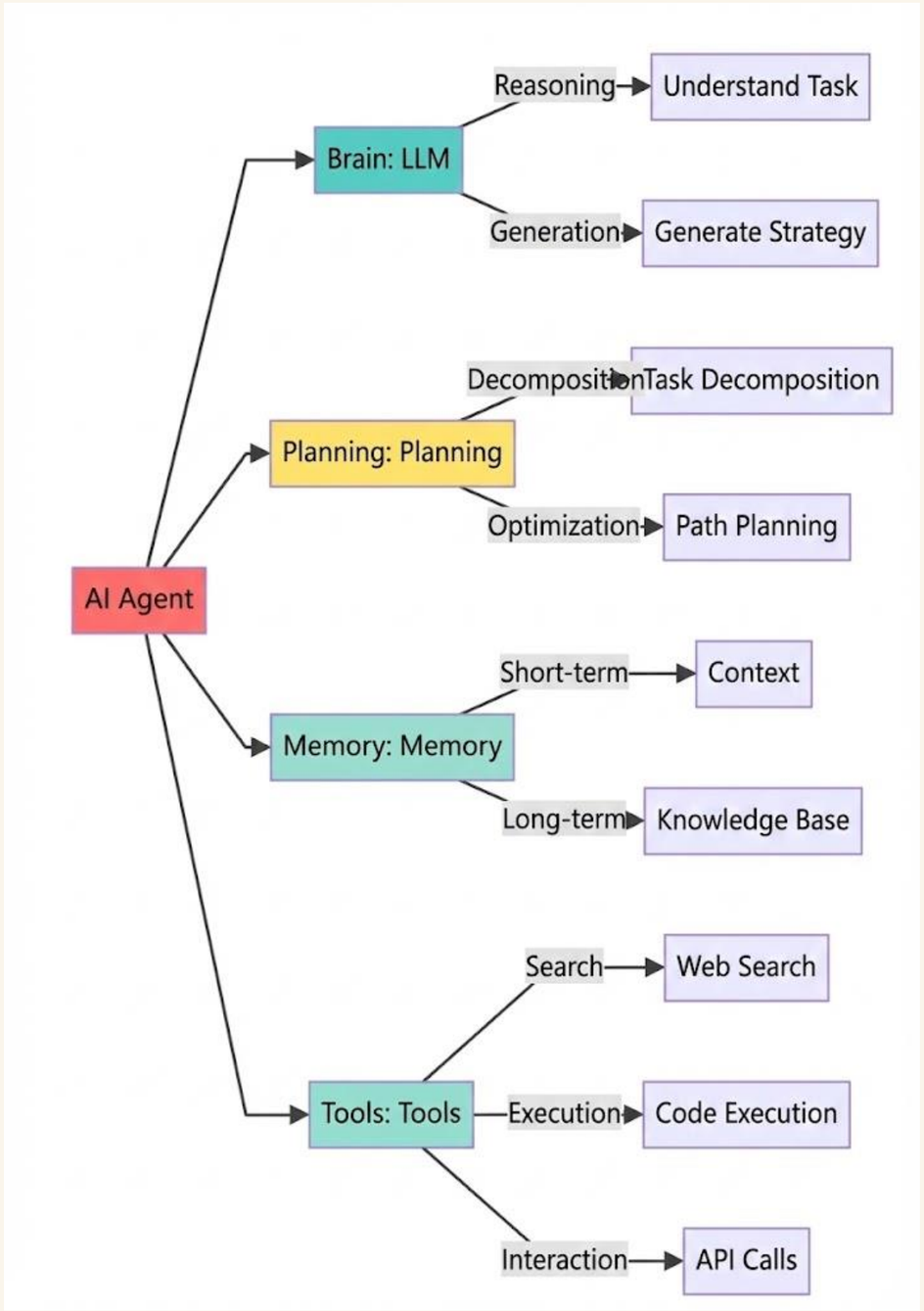


Single Step vs. React Agent

The Agent Loop

- Reason
- Act
- Feedback
- Adapt





Agent = LLM + Planning + Memory + Tools

Langchain and Langraph

What is Langchain?

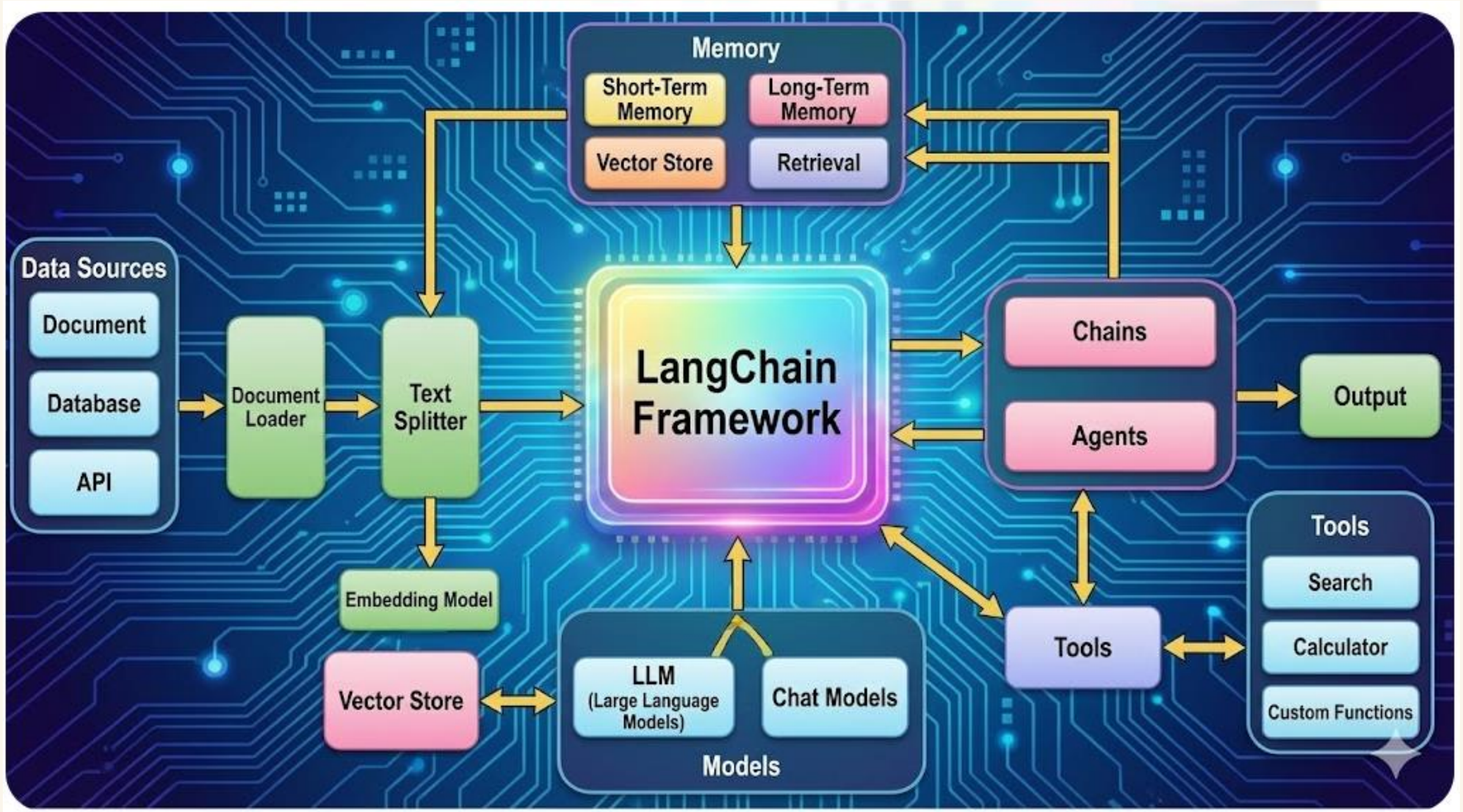
LangChain = a framework that lets large language models “get things done.”

Before, ChatGPT could only chat. With LangChain, you can let AI:



- Check the weather, browse the web
- Read files and write code
- Call your company’s internal APIs
- Decide on its own what actions to take

Langchain Framework

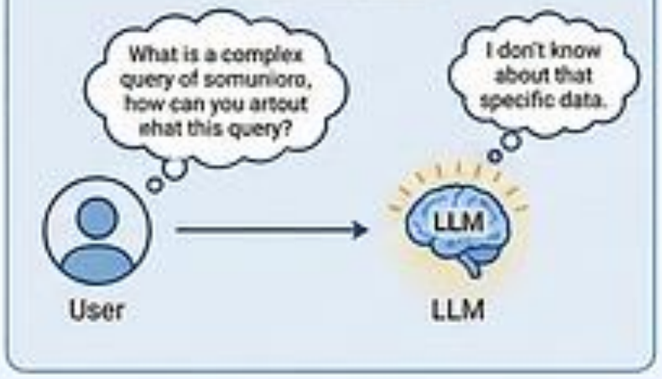


LangChain: A Beginner-Friendly Guide

LangChain: Your Bridge to Building AI Apps

Connect Large Language Models (LLMs) to the real world

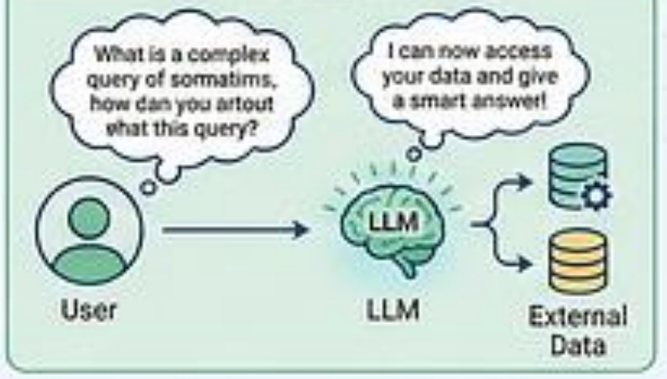
THE PROBLEM



LANGCHAIN: The Middleware Framework



THE SOLUTION



KEY BUILDING BLOCKS

Prompts & Templates
Structure instructions for the LLM. Use templates for reusability.

Chains
Combine multiple steps (Input -> LLM -> Output) into a sequence.

Memory
Remember past interactions for conversational context.

Agents & Tools
AI that can reason, plan, and use external tools (e.g., Search, Calculator).

POPULAR USE CASES FOR BEGINNERS

Chatbots
Build intelligent chatbots with memory and personality.

Document Q&A / RAG
Ask questions about your own PDFs, websites, or databases.

Summarization
Condense long text into concise summaries.

Source: Medium - LangChain Beginner Guides. Designed for clarity and learning.

Environment Setup

```
# Create a virtual environment (recommended)
python -m venv langchain_env
source langchain_env/bin/activate   # Mac / Linux

# langchain_env\Scripts\activate    # Windows


# Install LangChain core packages
pip install langchain langchain-core langchain-community


# Install model-specific integrations as needed
pip install langchain-openai       # OpenAI
pip install langchain-anthropic    # Claude

# Other commonly used packages
pip install python-dotenv          # Environment variable management
pip install langgraph              # Agent runtime (newer approach)
```

.env file

```
# OpenAI
OPENAI_API_KEY = sk-xxx

# Claude
ANTHROPIC_API_KEY = sk-xxx
```

Core Capabilities

Capability	Description	Example
Model Abstraction	One codebase can work with multiple large language models	Use GPT-4 today, switch to Claude tomorrow without changing the code
Tool Invocation	Allows AI to call external functions and services	AI helps you check stock prices, send emails, or operate databases
Memory Management	Enables AI to remember conversation history	Multi-turn conversations don't "forget" earlier context
Agent	Allows AI to make decisions autonomously	Give it a goal, and it plans how to complete it on its own

Four Core Components

Concept	What It Is	Analogy
Model	Large Language Model (LLM)	The brain
Prompt	Prompt / instruction template	The question format
Tool	External tools and functions	The hands
Agent	Intelligent agent	The manager that coordinates everything

User asks a question → Prompt formatting → Agent reasoning →
Tool invocation → Model generates an answer → Response returned to the user

C.R.A.F.T

🔍 **Context**: Provide background, details or sources in your prompt

🎭 **Role**: Specify who you want AI to “be” (e.g., expert, teacher, marketer)

👥 **Audience**: Specify who the response is for, better with details

📝 **Format**: Decide on the output style, length, or structure (list, story, table, etc.)

💬 **Task & Tone**: Clearly state the desired action and the preferred style or mood



https://www.instagram.com/p/DQvNpe3DKwF/

Prompt Template

Template Start-up

```
from langchain_core.prompts import PromptTemplate

# Define a template, using {} as placeholders
template = PromptTemplate.from_template(
    "Please use {language} to introduce {topic}, no more than 100 words."
)

# Fill in variables
prompt = template.format(language="English", topic="Artificial Intelligence")
print(prompt)

# Output:
# Please use English to introduce Artificial Intelligence, no more than 100 words.
```

Chat Prompt Templates

```
from langchain_core.prompts import ChatPromptTemplate

# Define a multi-turn chat prompt template
chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are a {role}. Explain complex concepts in a simple and easy-to-understand way."),
    ("human", "Please explain: {concept}"),
])

# Generate the message list
messages = chat_template.format_messages(
    role="Physics Professor",
    concept="Quantum Entanglement"
)

# Call the model
response = llm.invoke(messages)
print(response.content)
```

Tools

Tools are functions that AI can call.
Once tools are defined, the AI will decide **when to use them** and **how to use them** on its own.

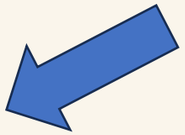
```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """
    Get weather information for a city.

    Args:
        city: Name of the city, such as "Beijing", ”New York"
    """

    # Mock weather data (in real projects, call a weather API)
    weather_data = {
        "Beijing": "Sunny, 25°C",
        ”New York": "Cloudy, 22°C",
    }

    return weather_data.get(city, "Weather data not found")
```



Write clear docstrings!
The AI relies on these descriptions to decide **when** to call the tool.

Bind Tools to the Model

After defining tools, you need to tell the model that these tools exist!

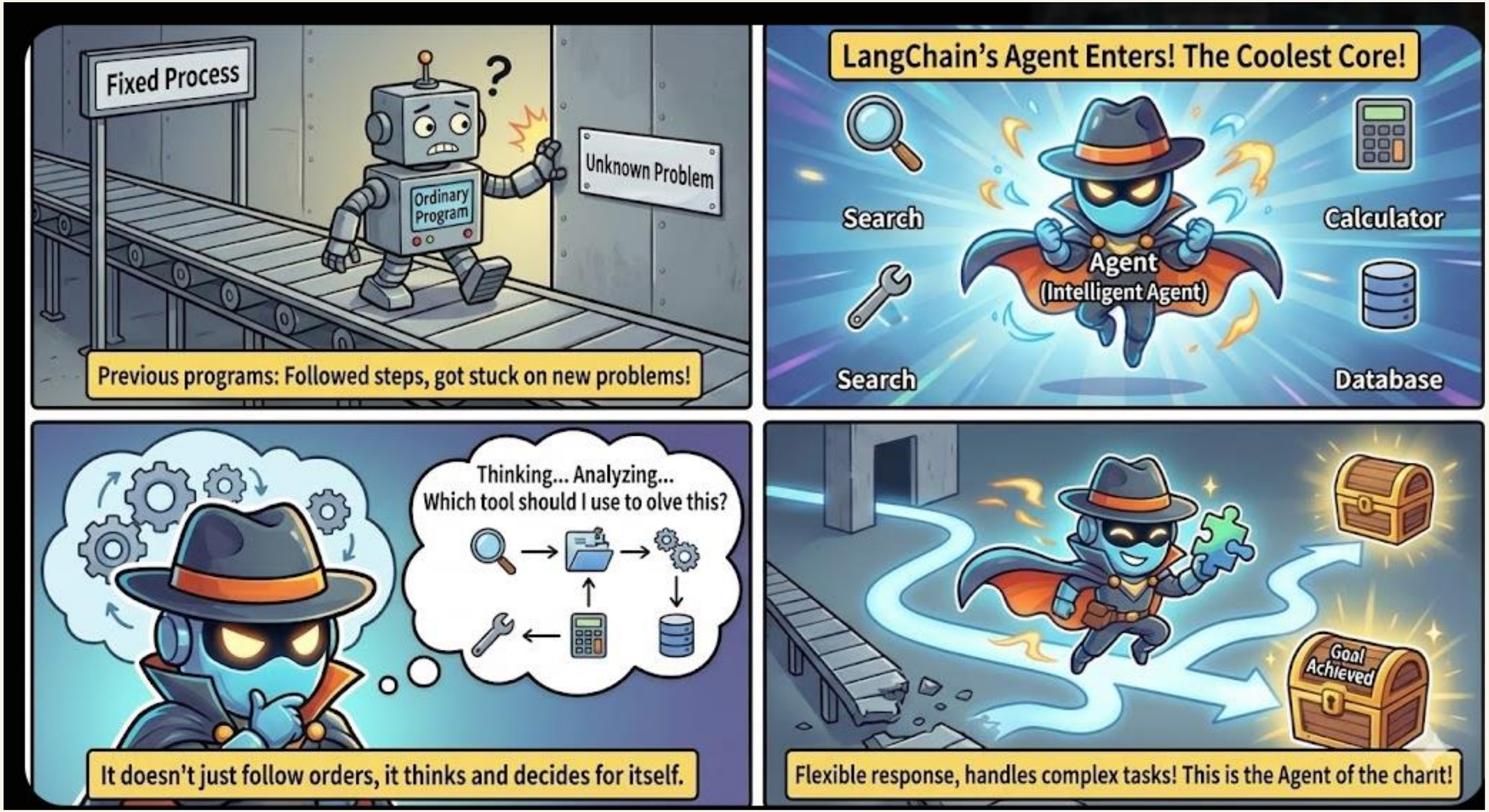
```
# Create a list of tools
tools = [get_weather, search_database, Calculator()]

# Bind tools to the model
llm_with_tools = llm.bind_tools(tools)

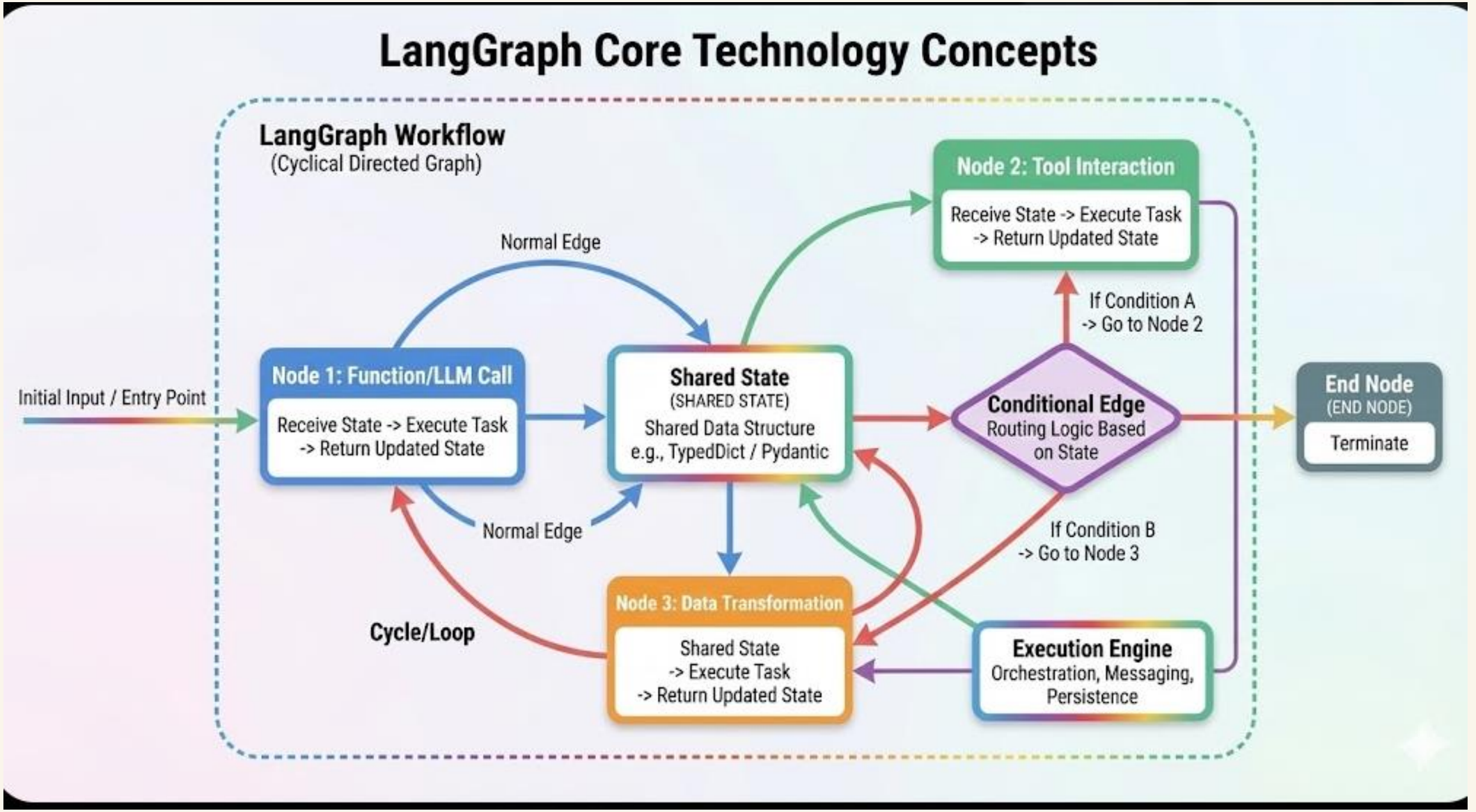
# Now the model knows these tools are available
response = llm_with_tools.invoke("What's the weather like in Beijing today?")

# Check whether the model decided to call any tools
if response.tool_calls:
    for tool_call in response.tool_calls:
        print(f"Model wants to call tool: {tool_call['name']}")
        print(f"Arguments: {tool_call['args']}")
```

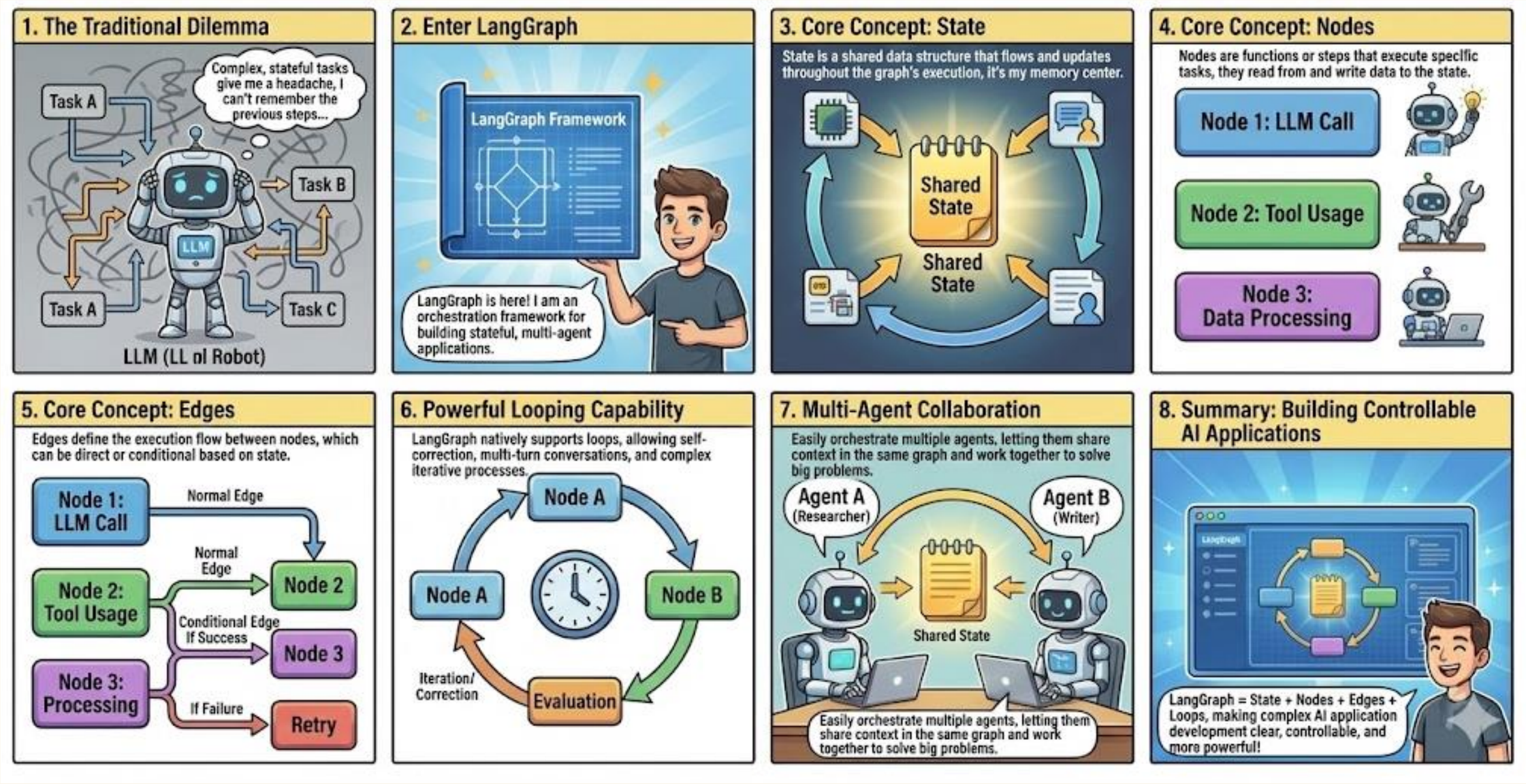
Agent Thinking Loop



What is LangGraph?


LangGraph Core Technology Concepts
LangGraph Workflow
(Cyclical Directed Graph)
Node 2: Tool Interaction
Receive State -> Execute Task
-> Return Updated State
Normal Edge
If Condition A
-> Go to Node 2
Initial Input / Entry Point
Node 1: Function/LLM Call
Receive State -> Execute Task
-> Return Updated State
Shared State
(SHARED STATE)
Shared Data Structure
e.g., TypedDict / Pydantic
Conditional Edge
Routing Logic Based
on State
End Node
(END NODE)
Terminate
Normal Edge
If Condition B
-> Go to Node 3
Node 3: Data Transformation
Shared State
-> Execute Task
-> Return Updated State
Cycle/Loop
Execution Engine
Orchestration, Messaging,
Persistence

What is LangGraph Framework?



Why do we Need LangGraph?

LangGraph is a framework for orchestrating AI workflows.
It lets you break complex AI tasks into small steps and then connect them together—like building with LEGO blocks.

Why Do We Need It?
Imagine you want to build a **customer support chatbot**:

1. **Determine** whether the user's question is a *technical issue* or a *sales question*
2. **Technical issues** are handled by a *Technical Agent*
3. **Sales questions** are handled by a *Sales Agent*
4. **If the answer quality isn't good enough**, loop back and improve it

Why LangGraph?

Workflows like this—with **branches, loops, and state**—quickly become messy when written with ordinary code.
LangGraph is designed specifically to handle these kinds of complex workflows.

LangChain vs. LangGraph

Feature	LangChain	LangGraph
Positioning	General-purpose LLM toolkit	Workflow orchestration framework
Workflow	Linear chains (A → B → C)	Graph-based (supports branching & loops)
State Management	Manual	Automatic
Best Use Cases	Simple Q&A, RAG	Complex agents, multi-step tasks

LangChain is for building components.
LangGraph is for assembling those components into complex systems.

LangGraph Three Core Concepts

State

- Shared data structure
- Passes information between nodes
- Defise using TypedDict
- Supports Reducer accumulation

Node

- Executes specific tasks
- Implemented as Python functions
- Can call LLM/tools
- Returns state updates

Edge

- Defines execution flow
- Normal Edge: Fixed path
- Conditional Edge: Dynamic routing
- Supports cycles/ branching

START → Node1 → Node2 → ... → END (State automatically passes rnden nodes)

LangGraph State

State is a **data structure shared across all nodes**. You can think of it as a "delivery package"—as it passes through each node, items are added to it or modified

Code block

Python

```python
from typing_extensions import TypedDict

class MyState(TypedDict):
    count: int          # Counter
    history: list[str]  # History record
```

LangGraph Node

Node is **what does the work**. Each node is a Python function that receives the current state and returns the fields that need to be updated.

Code block

Python

```python
def my_node(state: MyState):
    # Read current state
    current_count = state["count"]

    # Return fields to update (no need to return all fields!)
    return {
        "count": current_count + 1,
        "history": state["history"] + ["Executed once"]
    }
```

LangGraph Edge

Edge defines **how nodes connect to each other**, determining the execution order.

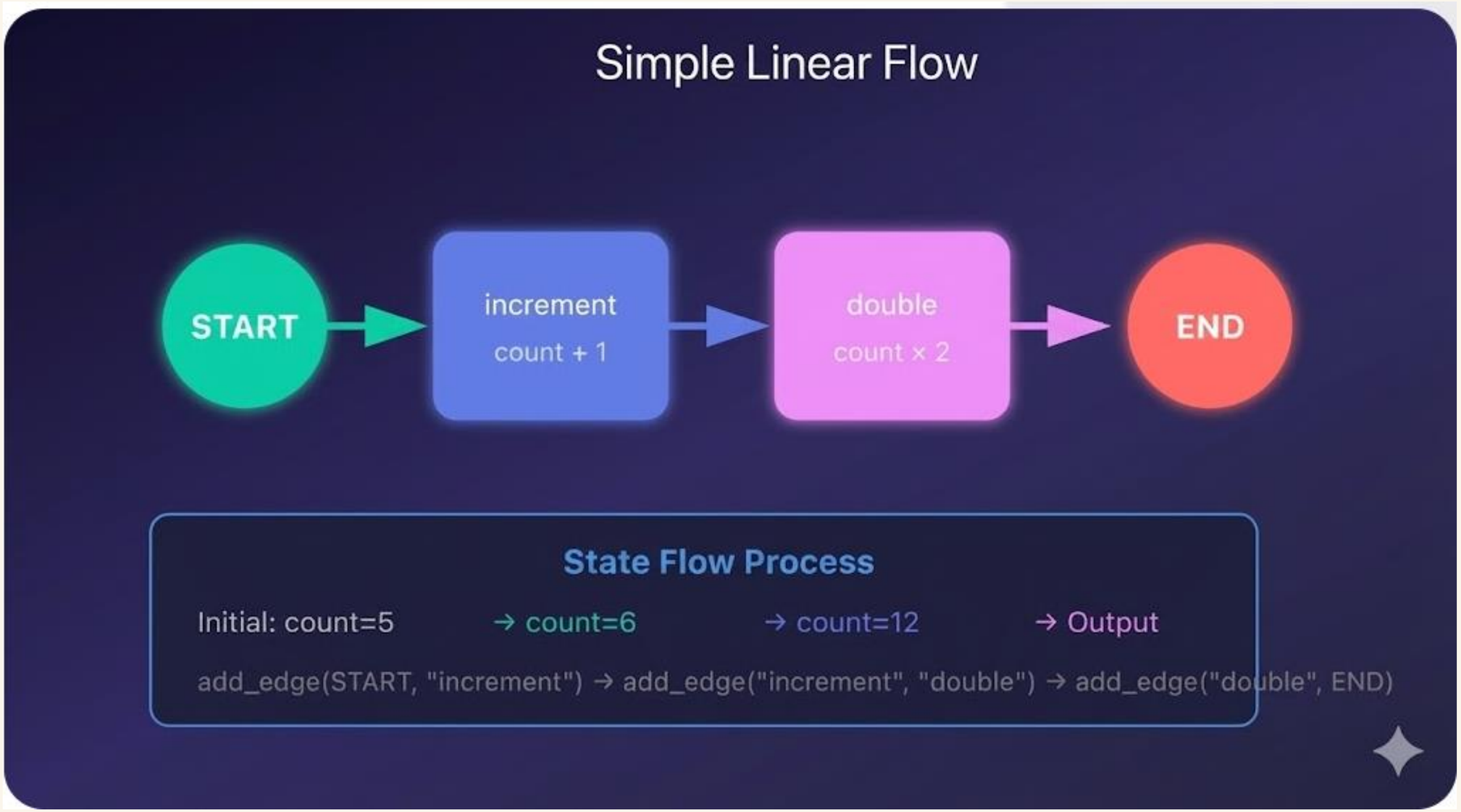
Code block

Python

```python
# Normal edge: Must execute B after A finishes
workflow.add_edge("A", "B")

# Conditional edge: Decide next step based on conditions
workflow.add_conditional_edges(
    "A",
    decision_function,
    {"Condition1": "B", "Condition2": "C"}
)
```

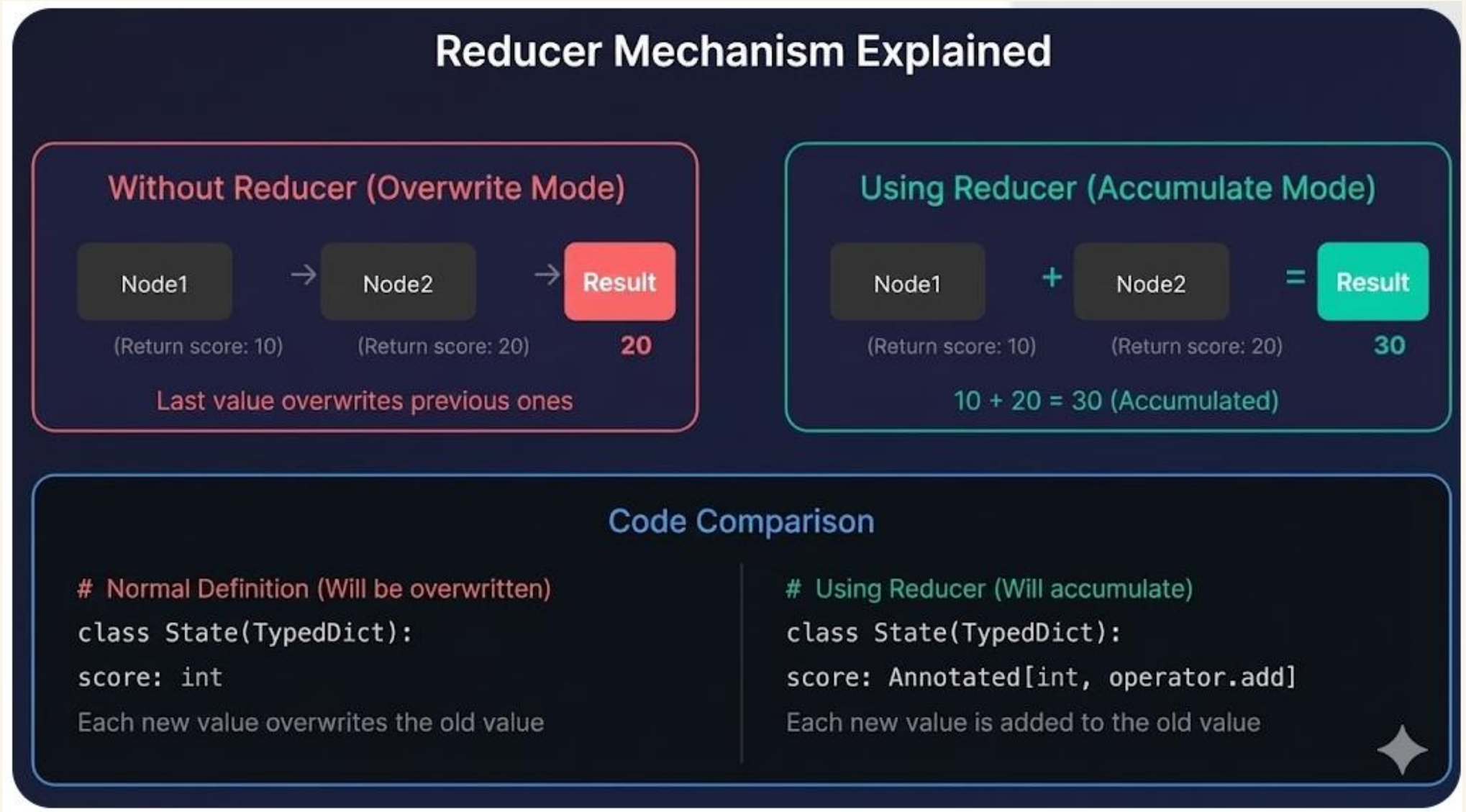
LangGraph Demo



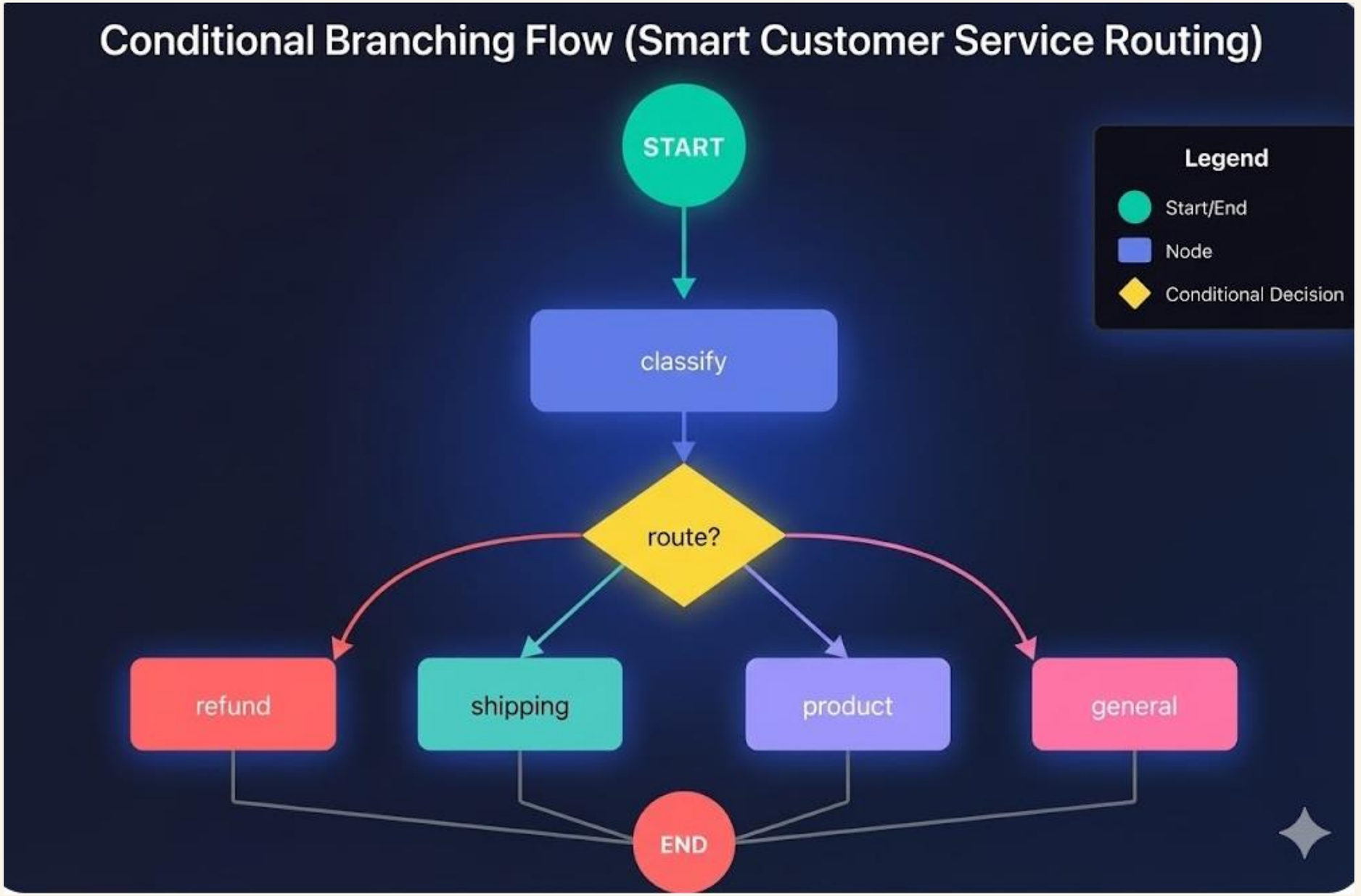
Code --- Demo

The Reducer Mechanism in LangGraph

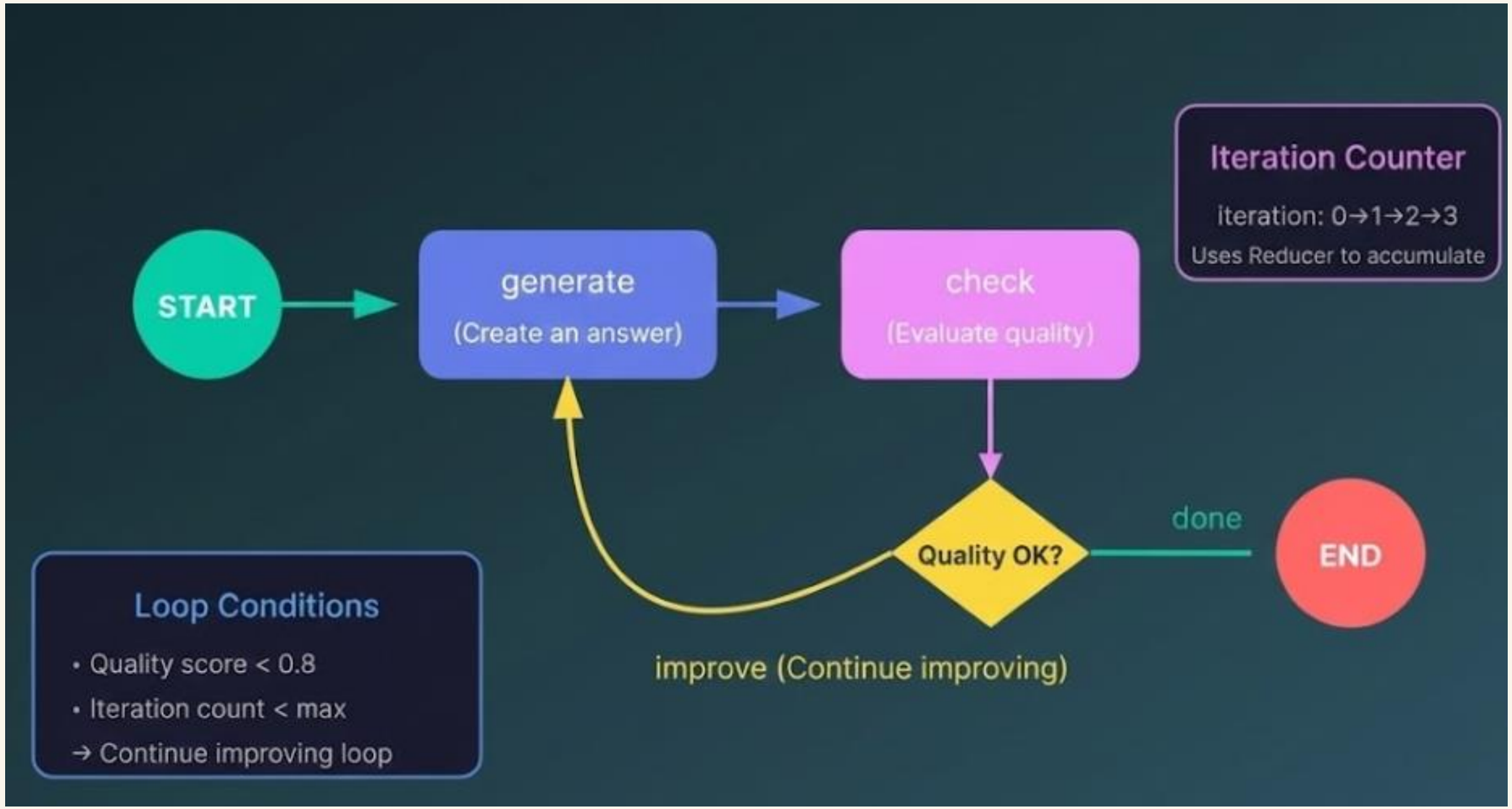
Why is state sometimes overwritten, and other times accumulated?



Conditional Branching



Loop Improvement Process



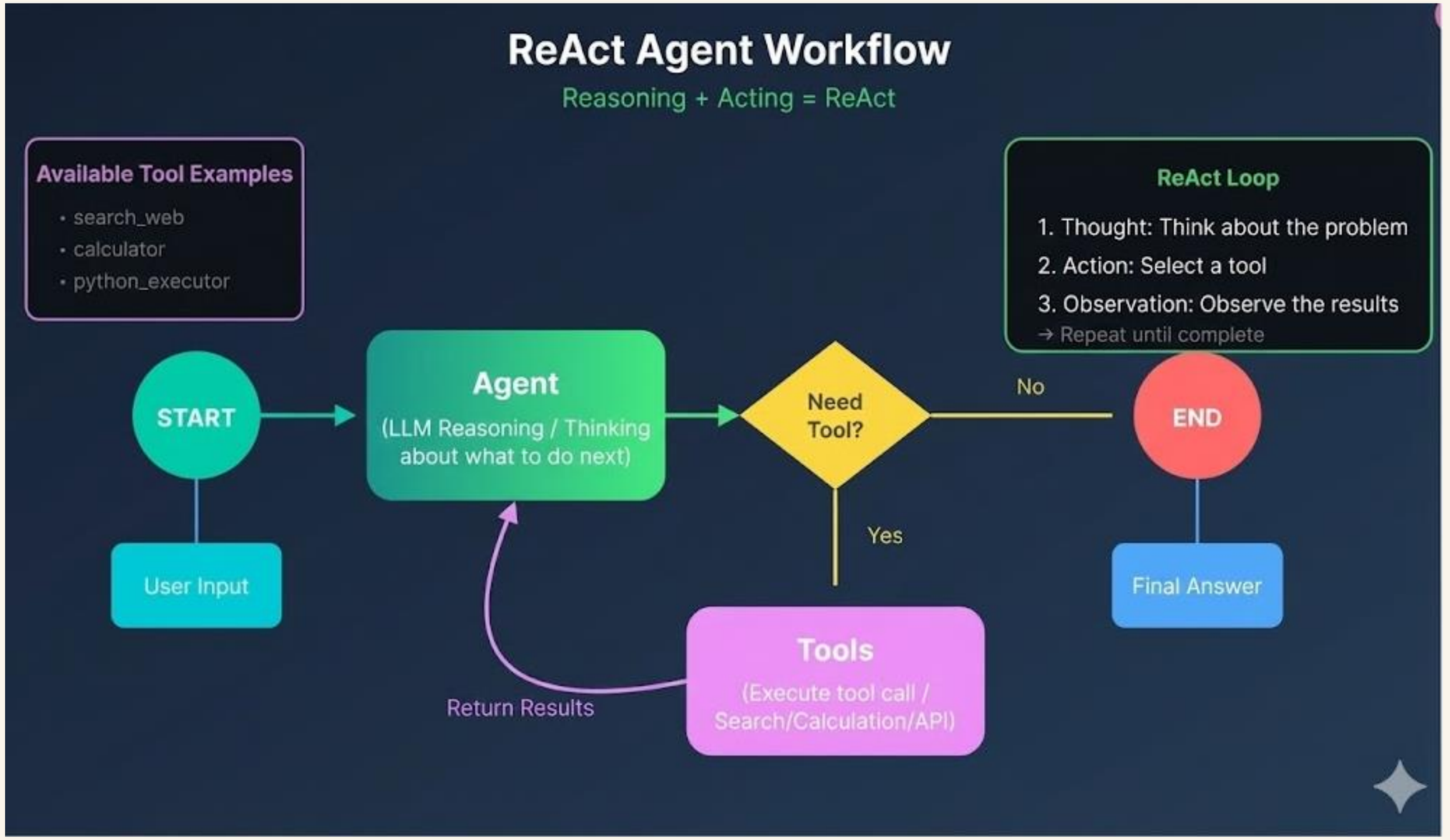
Key Takeaways

- **Conditional edges can point to previous nodes:** This creates a cycle or loop in the graph.
- **Must have an exit condition:** Without a condition (like "Quality OK"), the graph will loop infinitely.
- **Recommended to set a maximum iteration count:** This acts as a safety net (fallback) to prevent getting stuck if quality never improves.

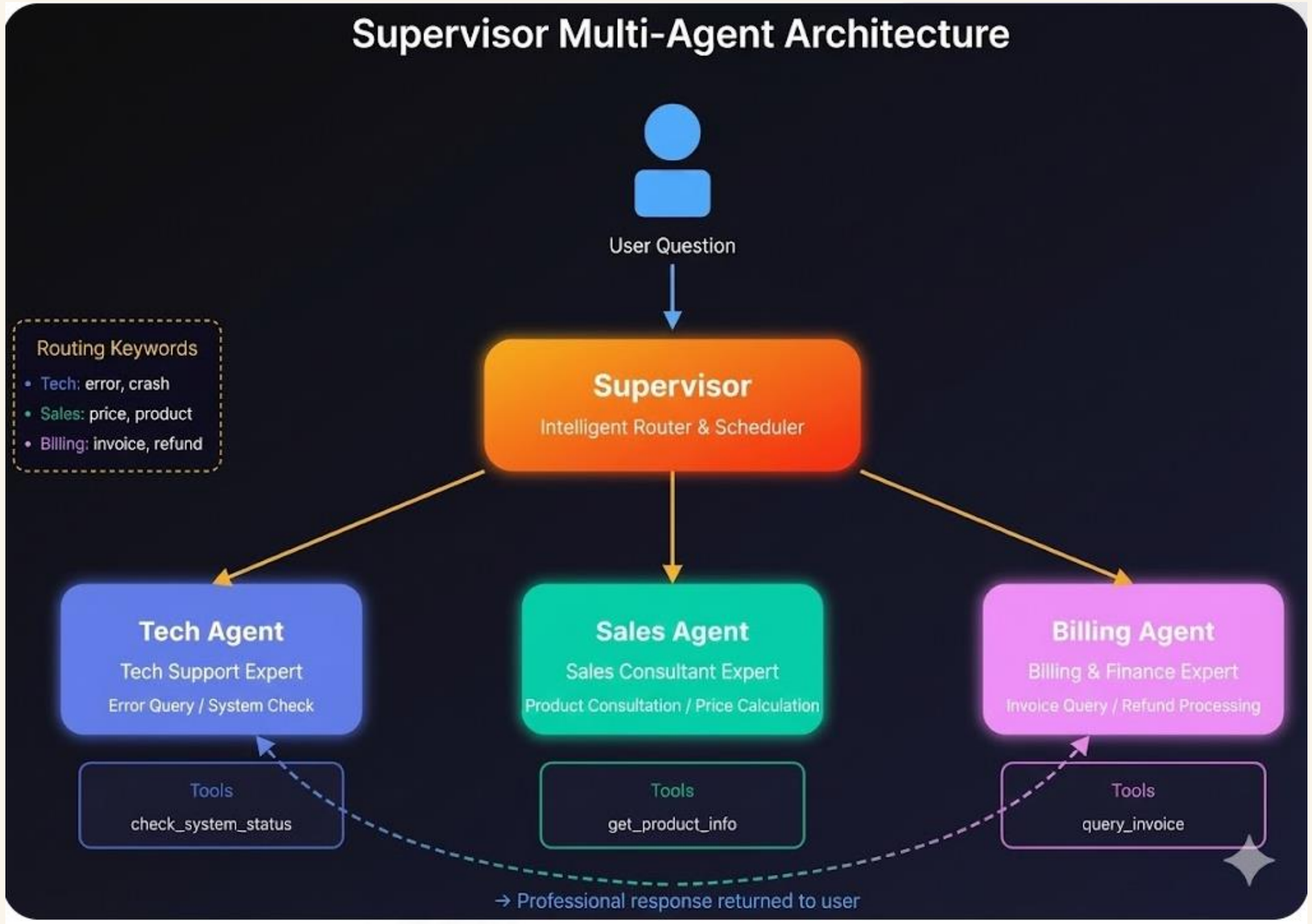
ReAct Agent Workflow

Workflow Steps

- **Thought:** The AI thinks about what needs to be done.
- **Action:** Selects and calls the appropriate tool.
- **Observation:** Observes the result returned by the tool.
- **Repeat:** Continues this cycle until the task is completed.



Multi-Agent Architecture

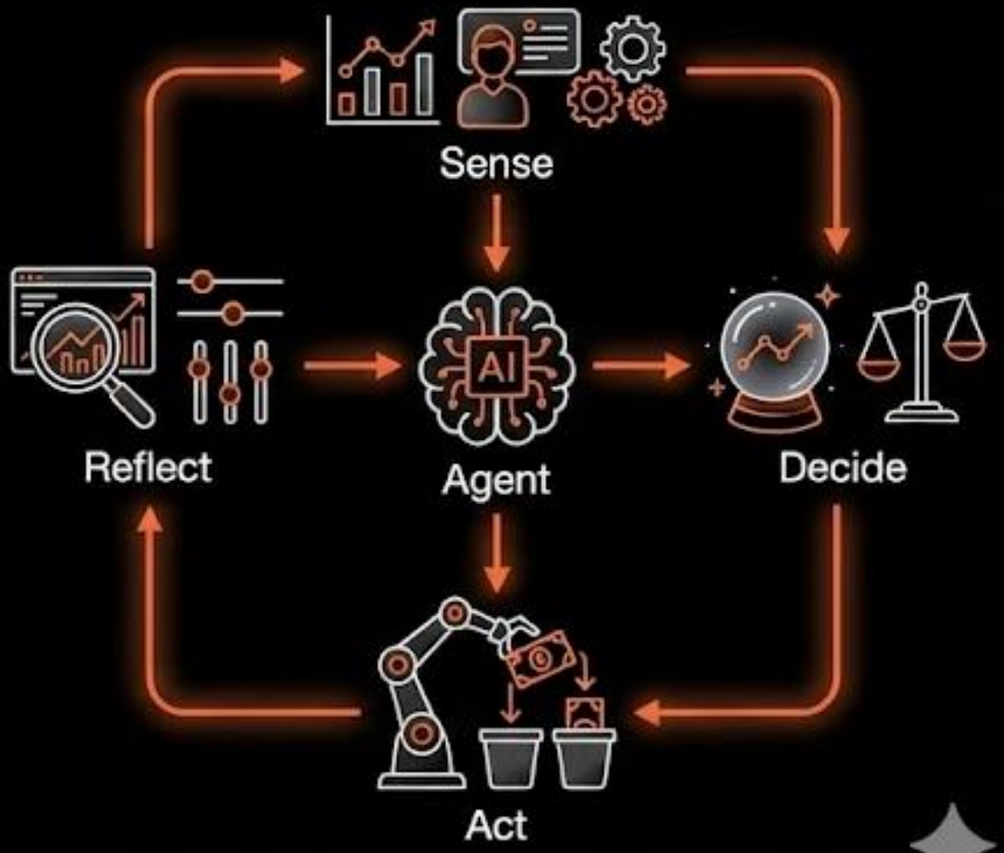


Core Concept: A Supervisor is responsible for routing and coordination, while multiple specialized Agents are responsible for execution.

Agentic Portfolio Rebalancing in Action

Feedback loop: Sense → Decide → Act → Reflect

- Agents observe: market data, investor profiles, constraints
- Plan: forecast market regime, assess drift
- Act: execute rebalance with cost/risk-awareness
- Reflect: review effect of action, adjust threshold



Agentic Finance: Rebalancing & Governance

The Agent's Feedback Loop & Safeguards



OBSERVE:
Market Data, Investor Profiles, Constraints



PLAN:
Forecast Regime, Assess Drift





EVALUATE:
Review Effects, Adjust Thresholds



EXECUTE:
Rebalance with Cost/Risk Awareness

Performance Metrics: Sharpe, Sortino, Drawdown, Tracking Error, Turnover

Ensures: Trust, Explainability, Regulatory Alignment

AI Agent Trends

Trend 1: Stronger Reasoning Ability

```
Current (2024):
  GPT-4: Good at language understanding and generation
  Limited reasoning ability (complex math, logic)

Future (2025-2026):
  OpenAI o1 series: Focused on reasoning
  DeepSeek-R1: Reinforcement learning reasoning
  Reasoning time↑, Reasoning accuracy↑

Impact:
  Agent can handle more complex tasks
  Reduced reliance on external tools
  Fewer errors and hallucinations
```

Trend 2: Multimodal Agent

```
Current: Mainly processes text

Future:
  Image understanding (recognizing charts, UI design)
  Video analysis (editing, content moderation)
  Voice interaction (real-time dialogue)
  3D environment operation (gaming, simulation)
Example Scenario: User uploads design sketch → Agent
recognition → Generate HTML/CSS → Automatic website
deployment
```

Trend 3: Personalization and Memory Enhancement

```
Current: Dialogue-level memory
Future:
  Cross-session long-term memory
  Personalized learning (understanding user habits)
  Proactive suggestions (not waiting for user query)
```

